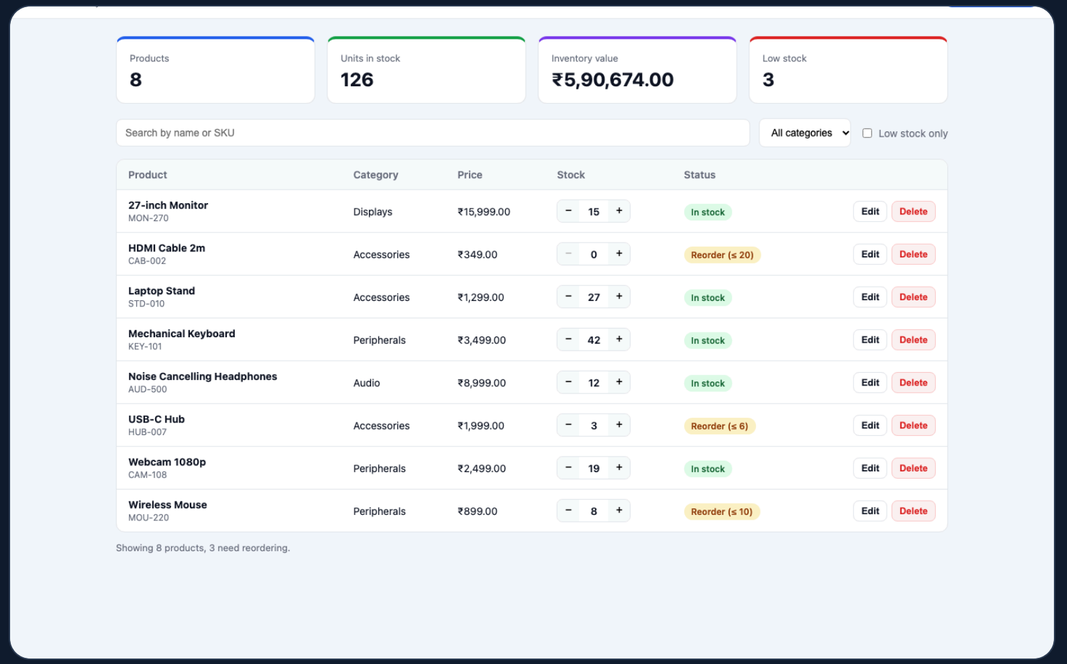
SESSION 21 · FINAL DEVOPS PROJECT

StockWise

An inventory app, taken from a laptop to a monitored Kubernetes cluster

Shubham Shah · Roll No. 10316



THE APPLICATION

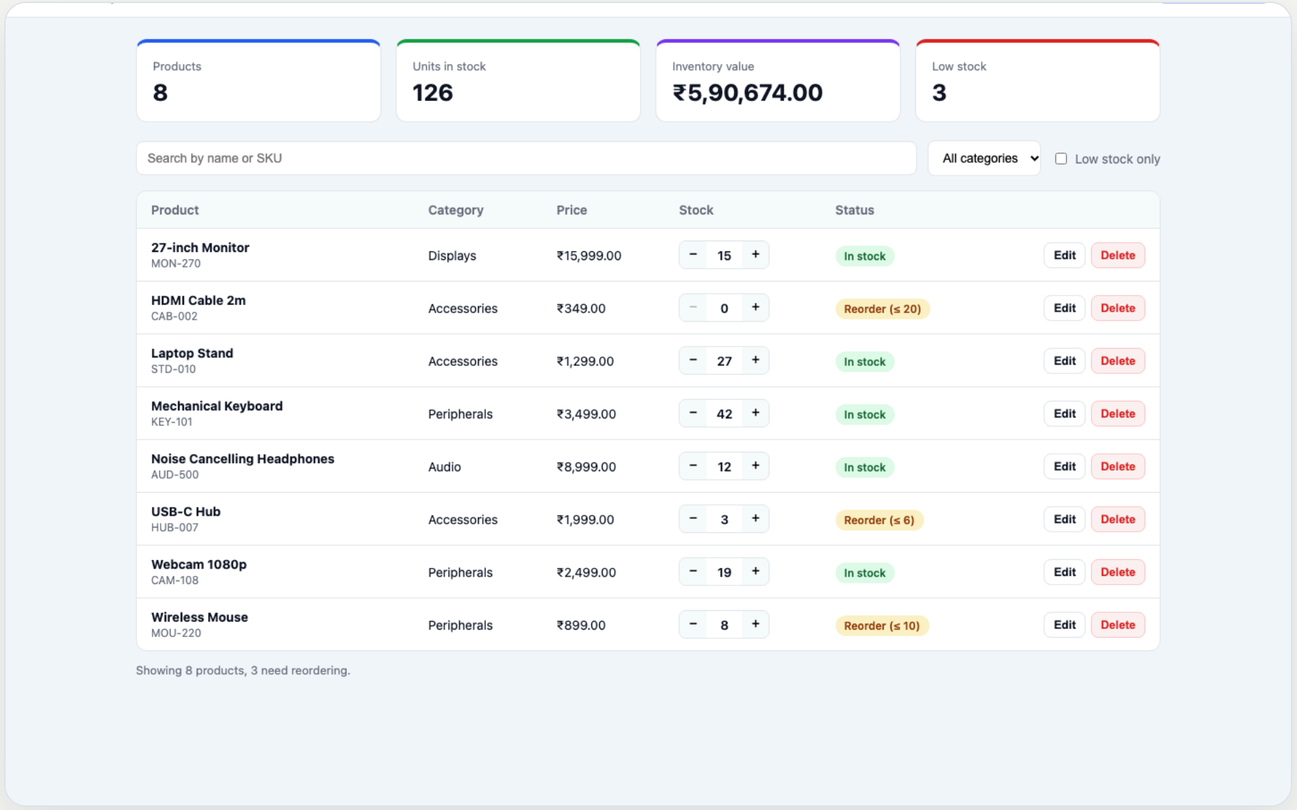
A store’s stock, at a glance

Staff add products, track stock as goods arrive and ship, and see at a glance what needs reordering.

FastAPI · React · PostgreSQL

7 product endpoints + health

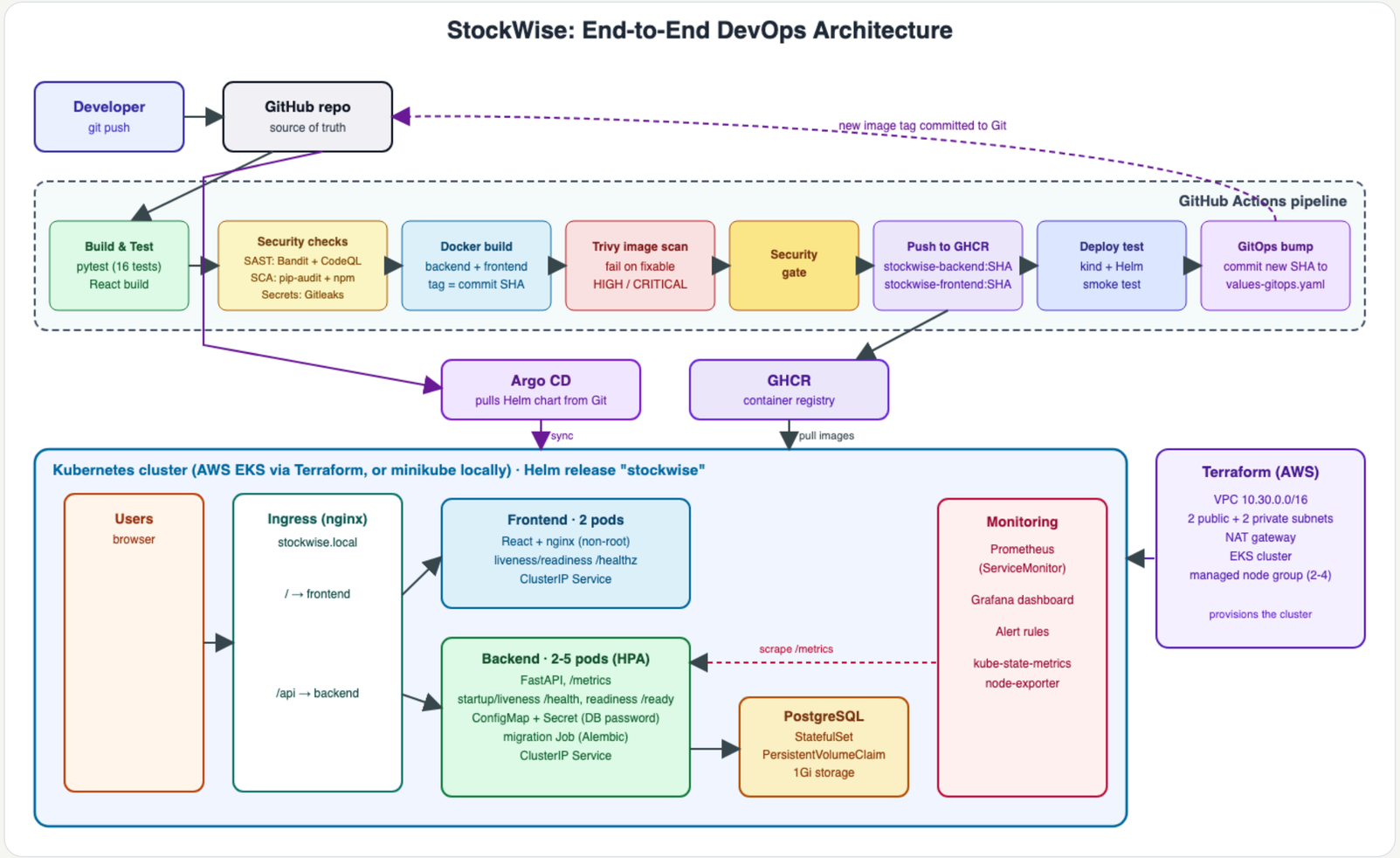
16 tests · 98% coverage



ARCHITECTURE

From a git push to a running cluster

The pipeline builds and checks every change. Argo CD then deploys whatever version Git says, and Prometheus watches it run.



TOOLS

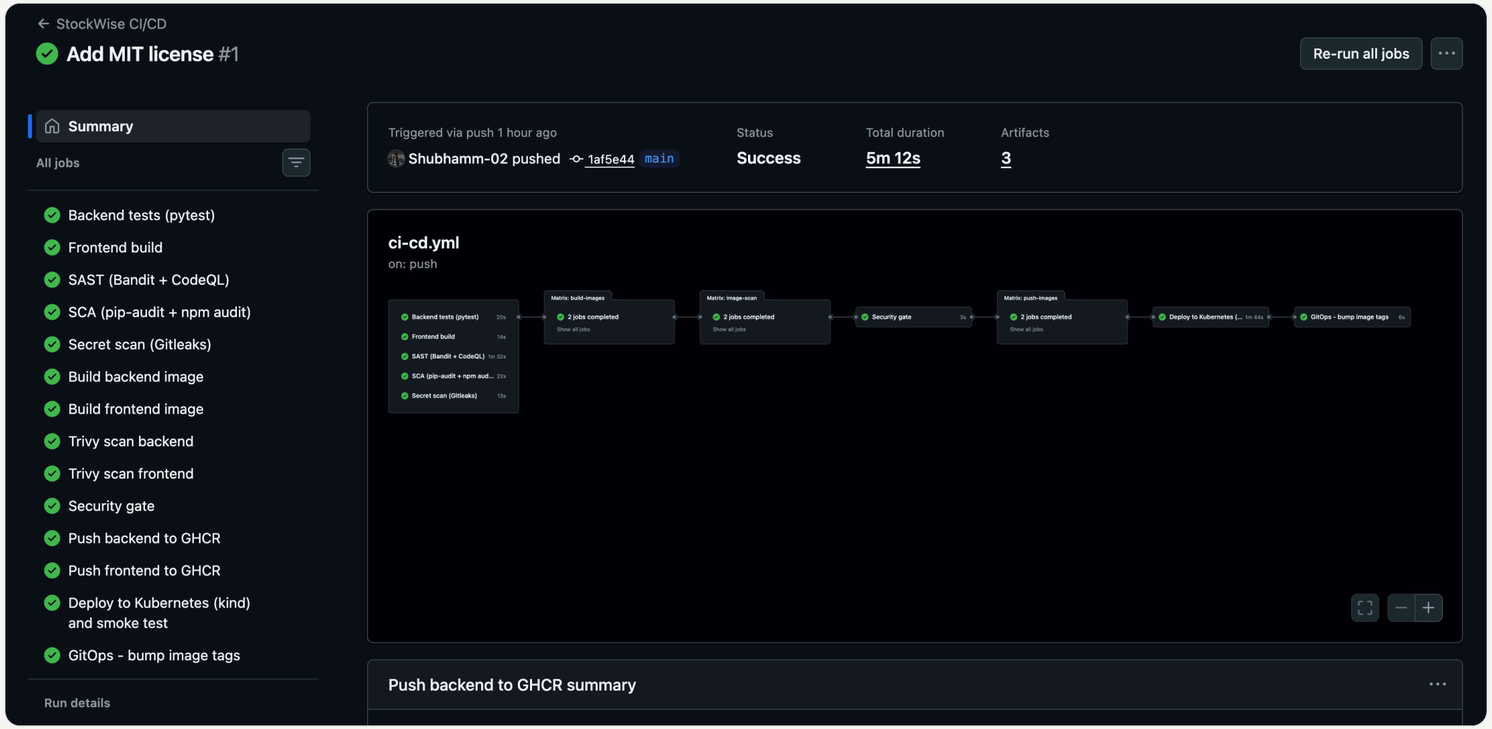
The stack, layer by layer

Layer	Tools
Application	FastAPI, SQLAlchemy, Alembic, React, Vite, PostgreSQL
Testing	pytest with coverage, SQLite test database
Containers	Docker multi-stage builds, Docker Compose, non-root images
CI/CD	GitHub Actions, GitHub Container Registry
Security	Bandit, CodeQL, pip-audit, npm audit, Gitleaks, Trivy
Platform	Kubernetes, Helm, nginx Ingress, HPA, metrics-server
Observability and GitOps	Prometheus, Grafana, Alertmanager, Argo CD
Infrastructure	Terraform, AWS VPC, AWS EKS

CI/CD

Every push runs 11 jobs before anything ships

Test | SAST · SCA | Secrets | Build | Trivy | **Gate** | Push | Deploy | GitOps



DEVSECOPS

Five checks, and any one can stop a release

- **SAST** · Bandit and CodeQL read our code
- **SCA** · pip-audit and npm audit check libraries
- **Secrets** · Gitleaks scans every file
- **Images** · Trivy blocks HIGH and CRITICAL
- **Gate** · nothing is pushed unless all pass

CAUGHT BY TRIVY

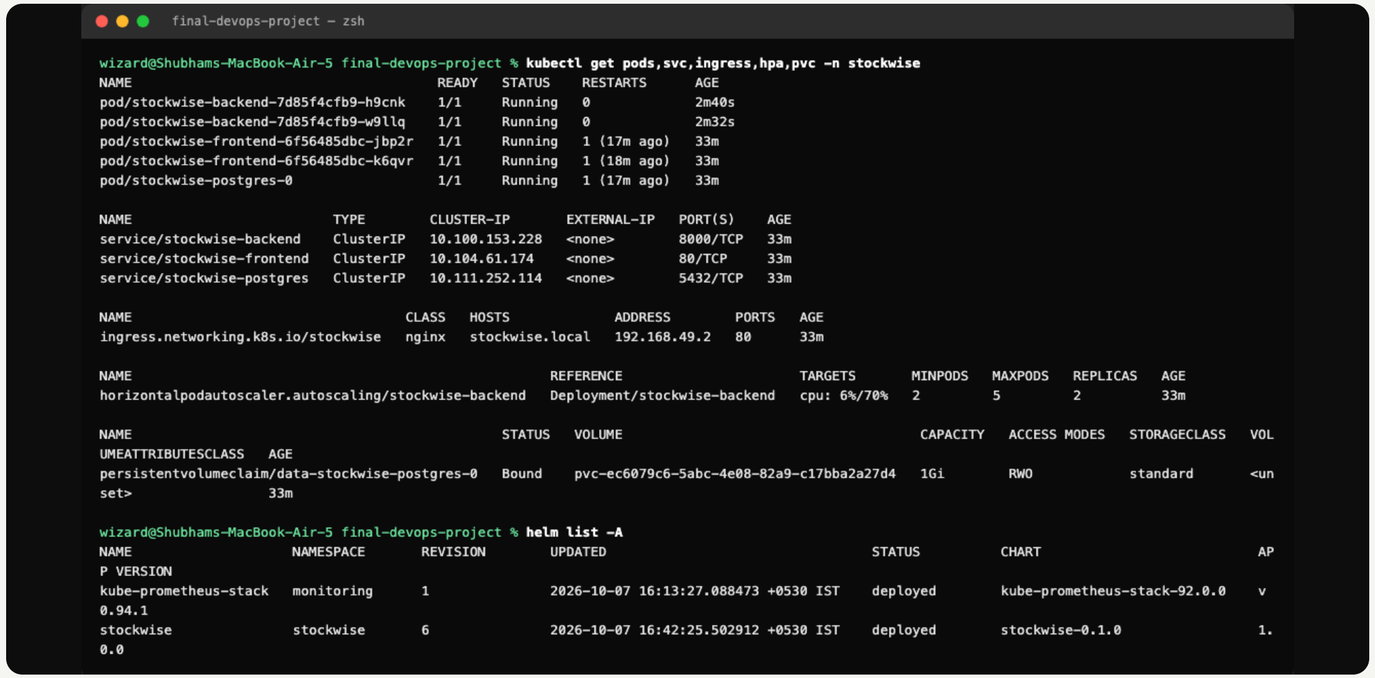
42 → 0

HIGH vulnerabilities in the frontend image. They came from old Alpine packages in the nginx base image, such as OpenSSL (CVE-2026-14456), not from my code. Patching the base image cleared all 42.

KUBERNETES AND HELM

One Helm chart deploys the whole app

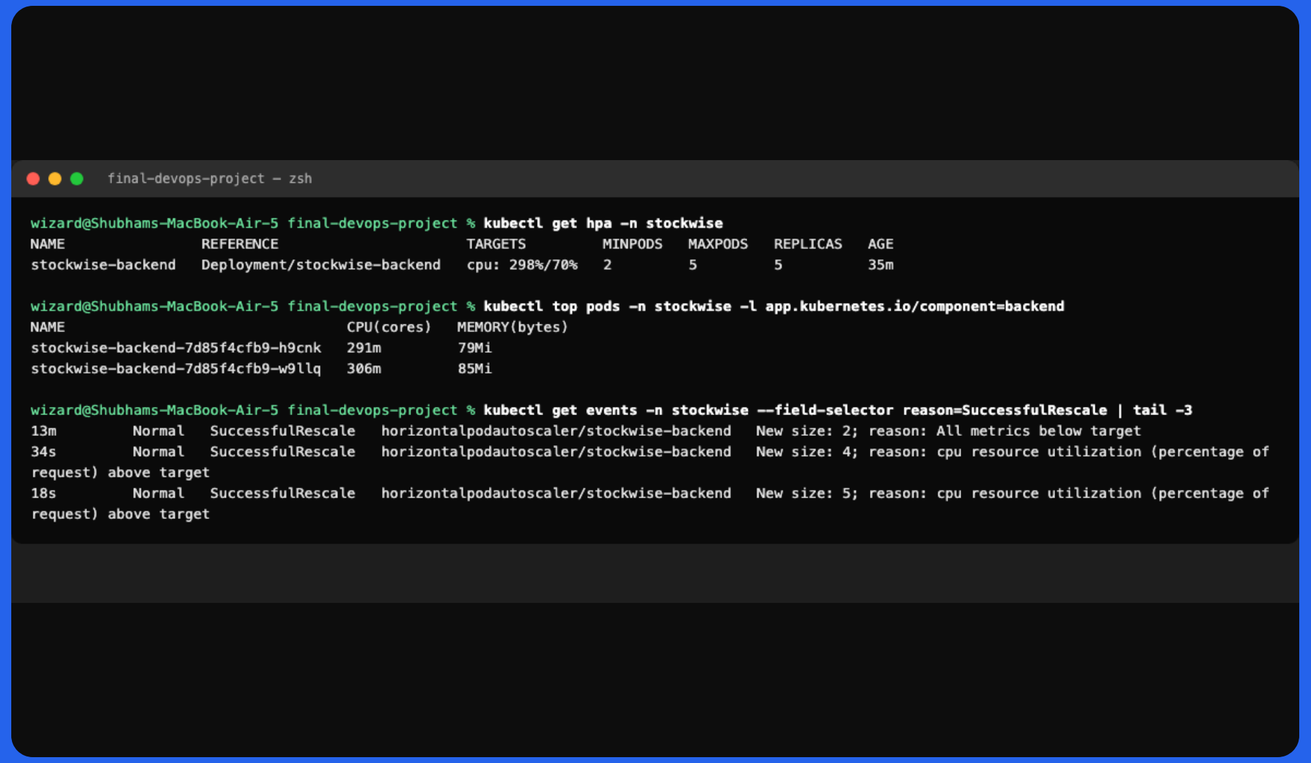
- **Backend** · 2 to 5 pods
- **Frontend** · 2 pods
- **PostgreSQL** · 1Gi volume
- **Ingress** · / and /api routes
- **Probes**, Secret, migration Job



AUTOSCALING UNDER LOAD

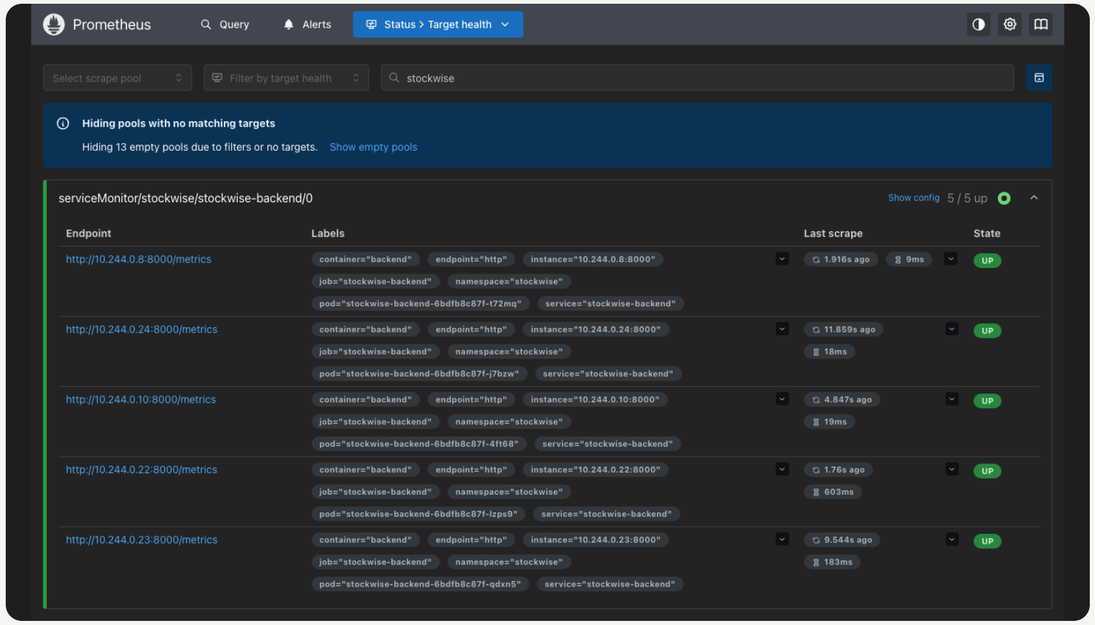
2 → 5 pods

During a load test, backend CPU reached 298% of its target. The autoscaler added pods up to the maximum of 5, then removed them once traffic stopped.

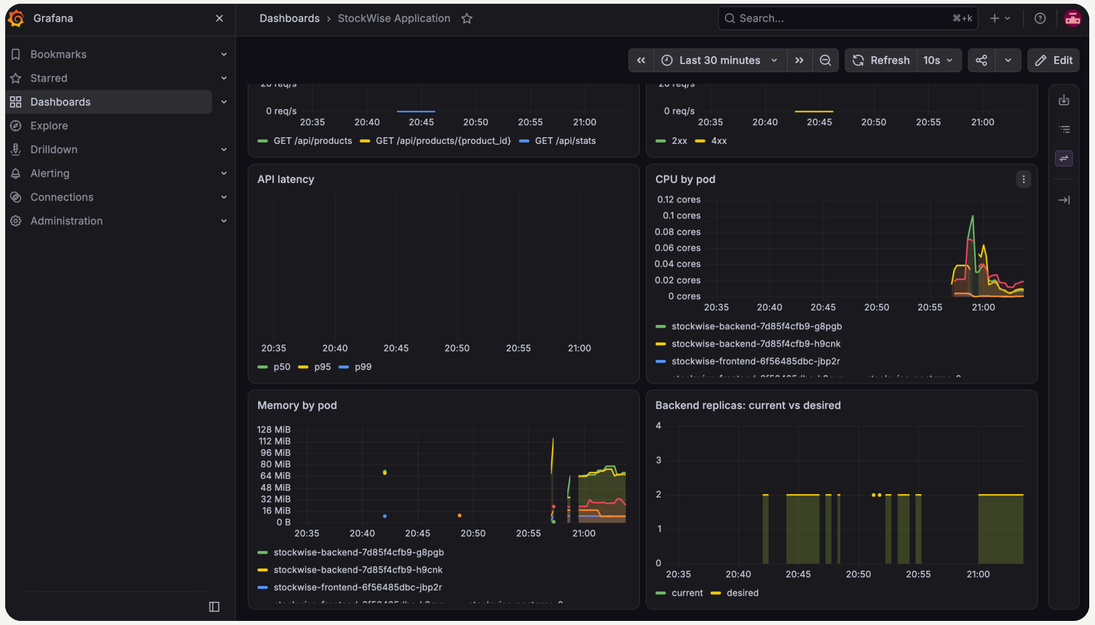


MONITORING

Prometheus scrapes every pod, Grafana shows it



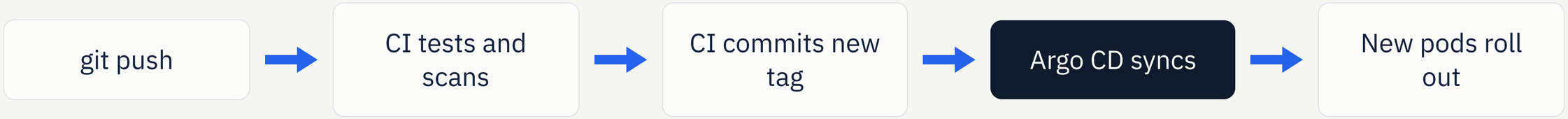
Prometheus finds all 5 backend pods through a ServiceMonitor.



A 12-panel Grafana dashboard covers traffic, errors, latency, CPU, memory and replicas.

GITOPS

Git decides what runs in the cluster



Self-heal

A manual change in the cluster is undone automatically.

Prune

Anything deleted from Git is deleted from the cluster.

Rollback

Reverting the tag commit redeploys the previous version.

INFRASTRUCTURE AS CODE

Terraform describes the AWS cluster

- **VPC** · 10.30.0.0/16 across 2 zones
- **Subnets** · 2 public and 2 private
- **NAT gateway** · outbound for private nodes
- **EKS** · Kubernetes 1.34 cluster
- **Node group** · 2 to 4 t3.medium

WORKFLOW

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
terraform destroy
```

No credentials in the code. State and terraform.tfvars stay out of Git.

TROUBLESHOOTING CHALLENGE

Four failures, introduced on purpose and fixed

Incident	Symptom	Root cause	Fix
Bad image tag	ImagePullBackOff	Tag v9.9.9 never existed	helm rollback
Selector typo	API returns HTTP 503	Service matched no pods	Fix the selector
Wrong DB password	Pod Running, 0/1 Ready	PostgreSQL rejects the login	Restore the Secret
Too much memory	Pod stuck Pending	64Gi asked, node has 8Gi	helm rollback

In every case the old pods **kept serving traffic**, because a new pod gets traffic only once it is Ready.

LESSONS LEARNED

What building this taught me

Readiness protects users

Broken versions never got traffic.

Scanners find real issues

The 42 findings came from a base image.

Build once, ship that

The scanned image is the deployed one.

One owner for migrations

A Job runs Alembic once per release.

Git is the record

Deploys and rollbacks are commits.

Destroy what you create

Cloud resources bill by the hour.

QUESTIONS?

Thank you

github.com/Shubhamm-02/stockwise-devops

Shubham Shah · Roll No. 10316